

第10章 泛型与元编程

建议学时:4-6 小时 | 难度:★★★★ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 理解 C 处理"类型无关代码"的两种笨办法:宏与 `void*` + 手工转换
- 掌握类型参数语法:泛型函数、泛型类型、实例化与类型推断
- 掌握约束系统:`any`、`comparable`、`~` 底层类型、联合约束、自定义约束
- 掌握泛型容器:泛型 `struct`、泛型接口与嵌套泛型
- 掌握 Go 1.21+ 标准库泛型工具:`slices` 与 `maps` 包
- 理解"方法不能带类型参数"等泛型边界,避免写出编译不过的代码
- 理解 Go 的元编程哲学:没有宏、没有模板特化,复杂生成交给代码生成

💡 从 C 到 Go:本章主题如何迁移

C 程序员对"类型无关代码"只有两件武器:①宏——`#define MAX(a,b) ((a)>(b)?(a):(b))`,展开时类型自适应,但宏没有类型检查、不能调试、参数多求值(`MAX(i++, j++)` 是灾难),一旦复杂就不可维护;②`void*` + 函数指针——`qsort/bsearch` 的路线,类型信息被抹掉,使用方必须手工做类型转换,编译器再也帮不了你,转错就是未定义行为。

Go 1.18 之前同样只有 `interface{}` 一件武器(第 6 章的 `any` 接口),底层还是类型断言,错误在运行时才炸。Go 1.18 引入的**泛型(类型参数)**是第三次革命:类型在编译期被"参数化",函数在调用处按实参类型自动实例化——既有宏的"类型自适应",又有完整编译期检查,还没有宏的求值副作用。

但 Go 的泛型刻意保持克制:没有 C++ 模板特化、偏特化、SFINAE;方法不能有类型参数;约束基于"类型集"。Go 的哲学是:元编程(模板、宏、反射代码生成)是复杂度的来源,能用普通代码解决的,绝不上元编程——这就是为什么本书记住一件事就够:优先写普通函数,只有确实需要"类型无关的算法与容器"时才上泛型。

📦 前置知识自查

- 第 6 章:接口与 `any(interface{})`,约束本质上就是接口的推广
- 第 4 章:高阶函数(`Filter/Map/Reduce` 思路)
- C 的宏与 `void*` 用法(对照基线)
- Go 1.21+(本章代码全部在 1.18+ 语法范围内)

🚀 本章学习路线

1. **第一阶段**:§10.1-§10.3,先写第一个泛型函数,理解实例化与推断
2. **第二阶段**:§10.4-§10.5,泛型类型与约束系统(重点:`~` 与 `comparable`)
3. **第三阶段**:§10.6-§10.8,标准工具包与边界,理解"克制"哲学
4. **验收标准**:能默写泛型 `Set` 与 `Filter/Map` 链式调用模板

本章知识导图

- §10.1 C 的困境:宏的求值陷阱与 void* 的类型抹除
- §10.2 类型参数语法:func F[T any](...),推断与显式实例化
- §10.3 约束基础:any、comparable 与"方法集约束"
- §10.4 泛型类型:泛型 struct 与泛型接口
- §10.5 类型集:~ 底层类型与联合约束(为什么 int 不等于 ~int)
- §10.6 标准工具:slices 与 maps 包的泛型函数
- §10.7 泛型边界:方法无类型参数、类型推断的盲区
- §10.8 Go 无宏:元编程哲学与代码生成预告

§10.1 C 的困境:宏与 void* 的代价

C 的写法	Go 的写法
<code>#define MAX(a,b) ((a)>(b)?(a):(b))</code> — 类型自适应但无检查、重复求值	<code>func Max[T comparable](a, b T) T</code> — 编译期检查、求值一次
<code>void qsort(void *base, ..., cmp)</code> — 类型抹除,用方手工转换,错即 UB	<code>slices.Sort([]T)</code> — 类型保留,编译器全程把关
宏展开进预处理阶段,与编译脱节	泛型实例化发生在类型检查阶段,错误在编译期报告
模板特化/重载(宏层面模拟)极易失控	无特化;算法对约束内所有类型一致行为
-Werror 也拦不住宏参数副作用	参数就是普通表达式,求值一次,无副作用陷阱

易错点 10-1:宏的重复求值是 C 的老坑

`MAX(a++, b)` 中 `a` 可能被求值两次,结果是未定义的;Go 泛型调用 `Max(a+1, b)` 中实参表达式只求值一次。这提醒我们:把泛型当作"带类型的函数",而不是"会展开的宏"。

§10.2 类型参数语法:第一个泛型函数

泛型函数在函数名后的方括号里声明类型参数: `func F[T 约束](参数...)`。调用时可以靠实参推断类型,也可以显式指定: `F[int](...)`。每个不同的实例化类型,编译器会生成对应版本的代码(类似于模板实例化,但发生在类型检查阶段,报错即编译错误)。

示例 10-1 泛型 Max 与推断

```

package main

import "fmt"

// Max 对任何可比较类型求最大值
func Max[T comparable](a, b T) T {
    if a > b { // comparable 保证可以使用 ==/!= 与比较
        return a
    }
    return b
}

// First 泛型切片工具:返回首元素与是否非空
func First[T any](s []T) (T, bool) {
    if len(s) == 0 {
        var zero T // 泛型零值:声明一个零值变量
        return zero, false
    }
    return s[0], true
}

func main() {
    fmt.Println(Max(3, 5))           // 推断为 int
    fmt.Println(Max(2.5, 1.8))       // 推断为 float64
    fmt.Println(Max("b", "a"))       // 推断为 string

    fmt.Println(Max[int](10, 20))    // 显式实例化
    fmt.Println(First([]string{"go", "lang"}))
    fmt.Println(First([]int{}))      // (0, false)
}

```

★ 重点:any 约束与 comparable 约束

- `any` (`interface{}` 的别名):对类型无任何要求,函数内只能用赋值、传参、类型断言
- `comparable` :要求类型支持 `==` 与 `!=` (内置可比较类型、可比较的结构体/指针/数组),可用在 `map` 的键、集合元素
- 约束写在方括号内,多个参数: `func Pair[A any, B comparable]()`

§10.3 约束:接口即约束

约束的底层机制就是接口:一个"类型集"。内置约束 `comparable` 是接口,自定义约束也是接口。约束可以要求方法(像普通接口),也可以要求底层类型与联合(用 `~` 与 `|`)。

■ 示例 10-2 方法集约束:自己定义约束

```

package main

import "fmt"

// Formatter 自定义约束:要求实现了 String 方法
type Formatter interface {
    String() string
}

// Describe 对任何实现 Formatter 的类型工作
func Describe[T Formatter](v T) string {
    return "描述: " + v.String()
}

type Point struct{ X, Y int }

func (p Point) String() string {
    return fmt.Sprintf("(%d, %d)", p.X, p.Y)
}

func main() {
    fmt.Println(Describe(Point{X: 1, Y: 2}))
}

```

⚠ 易错点 10-2:约束里的接口不能有类型参数

方法签名里不能出现类型参数——`func (s *Stack[T]) Push(v T)` 合法,但接口里写 `Set[T]` 的泛型方法是非法的(Go 明确禁止泛型方法)。这限制了"泛型接口"只能通过类型参数定义,不能通过方法实现多态。想突破时,先考虑是否能用接口 + 类型断言替代。

§10.4 泛型类型:容器类

泛型 struct、map、channel 都可以:类型参数声明在类型名后。最典型的应用是容器:泛型栈、泛型集合。

示例 10-3 泛型栈

```

package main

import "fmt"

// Stack 泛型栈:任意类型 T
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(v T) {
    s.items = append(s.items, v)
}

// Pop 弹出并返回:零值 + ok 判断空栈
func (s *Stack[T]) Pop() (T, bool) {
    n := len(s.items)
    if n == 0 {
        var zero T
        return zero, false
    }
    v := s.items[n-1]
    s.items = s.items[:n-1]
    return v, true
}

func (s *Stack[T]) Len() int { return len(s.items) }

func main() {
    var st Stack[string] // 显式实例化类型
    st.Push("第1条")
    st.Push("第2条")
    for st.Len() > 0 {
        if v, ok := st.Pop(); ok {
            fmt.Println(v)
        }
    }
}

```

§10.5 类型集:~ 底层类型与联合约束

数值运算(加法、乘法)无法用 comparable 表达——接口约束只能表达方法。于是 Go 引入"类型集"语法:用 | 联合多个类型,用 ~ 表示"底层类型匹配"。这是泛型里最容易绕晕的部分,但也是最有用的部分。

示例 10-4 数值工具:类型集约束

```

package main

import "fmt"

// Number 类型集:底层类型是整型或浮点型的类型
// ~int 表示"底层类型为 int 的类型",含 type MyInt int
type Number interface {
    ~int | ~int8 | ~int16 | ~int32 | ~int64 |
        ~uint | ~uint8 | ~uint16 | ~uint32 | ~uint64 |
        ~float32 | ~float64
}

// Sum 对任意数字切片求和
func Sum[T Number](xs []T) T {
    var total T // 零值起加
    for _, x := range xs {
        total += x
    }
    return total
}

type Money int // 底层类型是 int 的自定义类型

func main() {
    fmt.Println(Sum([]int{1, 2, 3, 4}))
    fmt.Println(Sum([]float64{1.5, 2.5}))
    fmt.Println(Sum([]Money{10, 20, 30})) // Money 也能用
}

```

示例 10-5 comparable 与 ~ 的对比

```

package main

import "fmt"

// 约束顺序:先满足方法,再满足类型集
type Ordered interface {
    ~int | ~float64 | ~string
}

// Min 需要有序类型:类型集约束
func Min[T Ordered](a, b T) T {
    if a < b {
        return a
    }
    return b
}

func main() {
    fmt.Println(Min(3, 2))
    fmt.Println(Min(1.9, 2.1))
    fmt.Println(Min("b", "a"))
    // 注意:Min(Money(1), Money(2)) 也合法,因为 ~int 包含 Money
}

```

⚠ 易错点 10-3: int 与 ~int 的区别

约束写 `int` 只匹配内建 `int` 本身;写 `~int` 匹配所有"底层类型是 `int`"的类型(包括 `type MyInt int` 与枚举式的 `type Weekday int`)。约束里 `~` 只能修饰有底层类型的类型,不能修饰接口与指针(如 `~*T` 非法)。规则:想让自定义整数类型也能调用,用 `~`;只想要内建类型,直接写类型名。

§10.6 标准工具包:slices 与 maps(Go 1.21)

Go 1.21 把泛型工具收编进标准库: `slices` 包提供排序、查找、裁剪、比较; `maps` 包提供克隆、合并、删除。以前要手写或用第三方库的通用操作,现在开箱即用。

示例 10-6 slices/maps 开箱即用

```
package main

import (
    "fmt"
    "maps"
    "slices"
)

func main() {
    nums := []int{5, 2, 8, 1, 9}
    slices.Sort(nums) // 泛型排序
    fmt.Println("排序:", nums)

    fmt.Println("二分查找 8 的位置:", slices.BinarySearch(nums, 8))

    // slices.Compact:去除相邻重复
    dup := []string{"a", "a", "b", "b", "c"}
    fmt.Println("去重(相邻):", slices.Compact(dup))

    m1 := map[string]int{"a": 1, "b": 2}
    m2 := maps.Clone(m1) // 浅拷贝
    m2["c"] = 3
    fmt.Println("原 map 受影响:", m1, "新 map:", m2)

    maps.DeleteFunc(m2, func(_ string, v int) bool { return v > 2 })
    fmt.Println("删除后:", m2)
}
```

§10.7 泛型边界:什么不能做

- **方法不能有类型参数**:接口方法签名中出现类型参数是非法的
- **类型参数不能用于结构体字段的接口断言目标之外**:如不能把 `T` 作为类型断言外的具体类型使用
- **没有特化与偏特化**:一个约束对应一种实现,无法为 `int` 与 `string` 提供不同实现(想要就写两个函数)
- **类型推断有限**:部分场景需显式写出类型参数(如约束无法从实参推出时)
- **泛型不能用于 go 声明外的 lambda 推断**:匿名函数不能带类型参数

示例 10-7 推断失败的典型场景与修复

```

package main

import "fmt"

// Pair 返回一对值:类型参数出现在返回类型而非参数
func Pair[A any, B any](a A, b B) (A, B) { return a, b }

// MustFirst 从 "值或错误" 的元组取出值
func MustFirst[T any](v T, err error) T {
    if err != nil {
        panic(err)
    }
    return v
}

func main() {
    // 推断成功:参数里出现 A/B
    fmt.Println(Pair("名字", 18))

    // 推断失败示例:返回值类型无法推断时必须显式写
    // r := New[User]() // 显式实例化
    x, y := Pair[int, string](1, "ok") // 显式写法
    fmt.Println(x, y)

    // 三元组解构的常见写法
    v := MustFirst(parse(42))
    fmt.Println("值:", v)
}

func parse(n int) (int, error) { return n * 2, nil }

```

工程建议:克制地使用泛型

三条纪律:①优先用普通函数与接口;②确实"类型无关且算法稳定"才上泛型(容器、排序、数学工具);③不要在 API 设计里堆砌三层嵌套的类型参数——可读性崩塌时,泛型的收益归零。标准库的判断法:如果 slices 包已经覆盖了需求,就别自己发明轮子。

§10.8 Go 无宏:元编程的克制哲学

C 有宏与模板,C++ 有完整的模板元编程;Go 两者都没有,也没有装饰器、注解、AOP。为什么?Go 团队的立场:元编程把"程序生成程序"的复杂性引入编译过程,报错晦涩、调试困难、社区碎片化。Go 的替代方案是**代码生成**:需要重复样板(如为每个类型生成 String 方法)时,写一个小程序生成源文件,交给 go generate 一键执行——生成的是普通 Go 代码,可读、可审、可测试。下一章(第12章)会完整演示 stringer 工具。

★ 重点:本章的元编程全景

- 宏/模板(编译期展开):Go 用泛型替代,类型安全
- 反射(运行期自省):Go 的 reflect 包,见第 11 章
- 代码生成(构建期生成源文件):go:generate,见第 12 章
- 注解/装饰器:AOP 式改写 — Go 没有,用中间件/包装函数替代(第 8 章 HTTP 中间件)

本章速查表

主题	要点
泛型函数	<code>func Max[T comparable](a, b T) T</code> ; 实参可推断, 也可显式 <code>Max[int](...)</code>
any 约束	无任何要求; 函数内只能赋值/断言
comparable	支持 <code>==/!=</code> ; map 键、集合元素、比较操作的前提
泛型类型	<code>type Stack[T any] struct</code> ; 方法接收者带 <code>[T]</code>
类型集约束	<code>~int ~float64 ~string</code> ; <code>~</code> 表示底层类型
自定义约束	约束就是接口; 可要求方法, 也可用类型集语法
slices 包	<code>Sort/BinarySearch/Compact/Clip/Equal</code> (Go 1.21+)
maps 包	<code>Clone/Copy/DeleteFunc/Equal</code> (Go 1.21+)
方法限制	方法不能有类型参数; 匿名函数不能泛型
无特化	无模板特化/偏特化; 行为对约束内类型一致
元编程	无宏/注解/AOP; 需要时用代码生成 (go generate)
使用纪律	普通函数优先; 容器与数学工具才泛型; 不堆嵌套参数

最小必会清单

- 能默写泛型 `Max[T comparable]` 与泛型 `Sum[~int|~float64]`
- 能默写泛型 `Stack[T any]` 完整类型与方法
- 能说出 `any` 与 `comparable` 的差别及适用场景
- 能解释 `~int` 与 `int` 在约束中的区别, 举出自定义类型示例
- 能写出方法集约束 (自定义 `Formatter` 接口)
- 能说出三个“泛型不能做”的限制
- 能默写 `slices.Sort + BinarySearch` 的调用
- 能解释“Go 为什么没有宏”及 `go generate` 替代方案

自测题

Q 1. `func Max[T comparable](a, b T) T` 为什么不能用 `any` 约束? 如果换成 `any` 会怎样?

✅ 参考答案

函数体里做了 `a > b` 的比较。`any` 不承诺任何操作, 比较运算符 (以及 `==/!=`) 在 `any` 上非法, 编译不过; `comparable` 保证 `T` 支持 `==` 与 `!=`, 但注意它 **不** 保证 `>` (有序性)——这就是为什么有序比较需要自定义类型集约束 (如 `Ordered` 约束) 而非 `comparable`。

Q 2. 约束里写 `int` 与写 `~int` 有何区别? `type MyInt int` 分别能否通过?

✅ 参考答案

写 `int` 只接受内建 `int`; `MyInt` 的底层类型是 `int` 但类型名不同,不能通过。写 `~int` 接受所有底层类型为 `int` 的类型(内建 `int`、`MyInt`、`type Weekday int` 等)。工程上处理自定义数值类型的约束几乎总是用 `~`。

Q 3. 下面的代码哪里错了?

✅ 参考答案

```
type Adder interface {  
    Add(other Adder) Adder // 编译错误?  
}  
  
func Add[T Adder](a, b T) T {  
    return a.Add(b).(T) // 这行也错  
}
```

第一处:泛型方法(接口方法参数/返回值用 `T`)被禁止——接口里不能声明类型参数,必须改为 `Add(other Adder) Adder` 且不涉及 `T`;第二处:类型断言 `.(T)` 的目标必须是具体类型而非类型参数(`Go` 限制,类型参数不能作为类型断言的目标)。正确做法是让约束接口包含足够方法,或用类型开关。

Q 4. Go 1.21 的 slices 包能替代哪些手写代码?举三个。

✅ 参考答案

`slices.Sort`(手写 `qsort`/插入排序/冒泡)、`slices.BinarySearch`(手写二分)、`slices.Compact`(手写相邻去重)、`slices.Equal`(手写逐元素比较)、`slices.Clip`(手写缩容)。这些是标准库对"泛型工具函数"的最佳实践,应优先使用。

Q 5. 为什么 Go 不提供模板特化?需要"不同类型不同实现"时怎么办?

✅ 参考答案

特化让同一个函数名在不同类型上行为不同,代码读者必须脑内展开分支,可读性与可预测性受损;`Go` 的立场是"简单优于复杂"。替代方案:①写两个不同的函数(如 `SortInts` 与 `SortStrings`);②用接口方法分发(把差异封装进类型的方法);③确实必须生成大量样板时,用代码生成(`go:generate`)而非语言级特化。

Q 6. 泛型容器 `Stack[T]` 的 `Pop` 为什么返回 `(T, bool)` 而非直接返回 `T`?写出该方法的实现。

✅ 参考答案

```
// 空栈无法产生一个合法的 T 值返回
func (s *Stack[T]) Pop() (T, bool) {
    n := len(s.items)
    if n == 0 {
        var zero T
        return zero, false
    }
    v := s.items[n-1]
    s.items = s.items[:n-1]
    return v, true
}
```

泛型没有"空值字面量",只能声明零值变量返回;用 bool 标志调用方区分"零值"与"真实零值",与通道接收的 ok 断言同构。

章末实验题:泛型工具库

📄 实验 10:泛型集合与函数式工具链

实验目标:实现一个泛型工具库:集合(Set)、高阶函数工具(Filter/Map/Reduce)、数值工具(Sum/Min/Max),全部基于类型参数与约束,覆盖本章全部知识点。

实验要求:

- 任务 1:Set[T comparable]:Add/Contains/Remove/Len,基于 map[T]struct{}(覆盖:\$10.2 泛型类型)
- 任务 2:Filter[T any](s, func(T) bool) []T 与 Map[T, U any] 转换(覆盖:\$10.2 多类型参数)
- 任务 3:Reduce[T, U any] 归约,支持"从空切片返回初始值"(覆盖:\$10.2 零值泛型)
- 任务 4:Sum/Product 用类型集约束 Number(覆盖:\$10.5 ~ 与 |)
- 任务 5:Min/Max 用自定义 Ordered 约束(覆盖:\$10.3 自定义约束)
- 任务 6:演示 type Money int 也能参与 Sum/Min(覆盖:\$10.5 ~ 的意义)
- 任务 7:与 slices/maps 标准包互操作:Sorted 返回排序切片(覆盖:\$10.6)
- 任务 8:实验一个"编译不过的泛型方法"并解释原因(覆盖:\$10.7 边界)

实验环境:Go 1.21+;先手写再用 gofmt;最后跑 go vet 确认无告警。

第一步 定义 Number 与 Ordered 两个约束接口

第二步 实现 Set[T comparable](内嵌 map,方法全在指针接收者上)

第三步 实现 Filter/Map/Reduce 三个纯函数(不动原切片,返回新切片)

第四步 实现 Sum/Product(类型集)与 Min/Max(Ordered)

第五步 在 main 中组合:构造集合、过滤、映射、归约、求和,全链路打印

✔ 参考答案要点

```
// 实验 10:泛型工具库—集合 + 高阶函数 + 数值工具
package main

import (
    "fmt"
    "slices"
)

// 任务 4:数值类型集约束($10.5)
type Number interface {
    ~int | ~int8 | ~int16 | ~int32 | ~int64 |
    ~uint | ~uint8 | ~uint16 | ~uint32 | ~uint64 |
    ~float32 | ~float64
}

// 任务 5:有序类型约束($10.3 自定义约束)
type Ordered interface {
    ~int | ~float64 | ~string
}

// ===== 任务 1:泛型集合($10.2 泛型类型)=====
type Set[T comparable] struct {
    m map[T]struct{} // 空结构体作占位
}

func NewSet[T comparable]() *Set[T] {
    return &Set[T]{m: make(map[T]struct{})}
}

func (s *Set[T]) Add(v T)      { s.m[v] = struct{}{} }
func (s *Set[T]) Remove(v T)   { delete(s.m, v) }
func (s *Set[T]) Contains(v T) bool {
    _, ok := s.m[v]
    return ok
}

func (s *Set[T]) Len() int { return len(s.m) }

// Elements 返回全部元素切片($10.6:slices 包互操作)
func (s *Set[T]) Elements() []T {
    out := make([]T, 0, len(s.m))
    for v := range s.m {
        out = append(out, v)
    }
    return out
}

// Sorted 返回排序后的元素
func (s *Set[T]) Sorted() []T {
    es := s.Elements()
    slices.Sort(es) // 标准库泛型排序($10.6)
    return es
}

// ===== 任务 2/3:高阶函数工具($10.2 多类型参数)=====
// Filter 保留满足条件的元素,不修改原切片
```

```

func Filter[T any](s []T, keep func(T) bool) []T {
    out := make([]T, 0, len(s))
    for _, v := range s {
        if keep(v) {
            out = append(out, v)
        }
    }
    return out
}

// Map 类型转换: []T → []U
func Map[T, U any](s []T, f func(T) U) []U {
    out := make([]U, 0, len(s))
    for _, v := range s {
        out = append(out, f(v))
    }
    return out
}

// Reduce 归约: 用初始值起步, 依次折叠
func Reduce[T, U any](s []T, init U, f func(U, T) U) U {
    acc := init
    for _, v := range s {
        acc = f(acc, v)
    }
    return acc
}

// ===== 任务 4/5: 数值与有序工具($10.5) =====
func Sum[T Number](s []T) T {
    return Reduce(s, 0, func(acc, v T) T { return acc + v })
}

func Product[T Number](s []T) T {
    return Reduce(s, 1, func(acc, v T) T { return acc * v })
}

func Min[T Ordered](s []T) T {
    if len(s) == 0 {
        panic("空切片求最小值")
    }
    m := s[0]
    for _, v := range s[1:] {
        if v < m {
            m = v
        }
    }
    return m
}

func Max[T Ordered](s []T) T {
    if len(s) == 0 {
        panic("空切片求最大值")
    }
    m := s[0]

```

```

    for _, v := range s[1:] {
        if v > m {
            m = v
        }
    }
    return m
}

// ===== main:全链路演示 =====
type Money int // 底层类型是 int 的自定义类型($10.5 ~ 的意义)

func main() {
    // 集合基本操作(任务 1)
    s := NewSet[int]()
    for _, v := range []int{3, 1, 4, 1, 5, 9, 2, 6} {
        s.Add(v)
    }
    fmt.Println("集合:", s.Elements(), "大小:", s.Len())

    // 排序互操作(任务 7)
    fmt.Println("排序后:", s.Sorted())

    // 过滤 + 映射 + 归约链(任务 2/3)
    nums := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
    evens := Filter(nums, func(v int) bool { return v%2 == 0 })
    doubled := Map(evens, func(v int) int { return v * 10 })
    total := Reduce(doubled, 0, func(acc, v int) int { return acc + v })
    fmt.Println("偶数:", evens, "x10:", doubled, "合计:", total)

    // 数值工具 + 自定义类型(任务 4/6)
    prices := []Money{30, 15, 75, 40}
    fmt.Println("价格合计:", Sum(prices), "最低:", Min(prices), "最高:", Max(prices))

    // 字符串有序比较(任务 5)
    words := []string{"pear", "apple", "banana"}
    fmt.Println("字典序最小:", Min(words))

    // 任务 8 思考:接口里写 func Add(T) T 无法编译—
    // Go 禁止泛型方法;要"泛型 + 多态"请用类型参数而非方法签名
    fmt.Println("=== 全部工具工作正常 ===")
}

```

设计说明:实验代码覆盖本章全部任务点:Set 用 `map[T]struct{}` 演示"空结构体零开销占位"这一经典惯用法;Filter/Map/Reduce 验证多类型参数与"返回新切片、不动原数据"的值语义;Sum/Product 通过类型集约束让 Money 这类自定义整数类型自动可用——这正是 ~ 的价值;Min/Max 用 Ordered 约束且对空切片显式 panic, 把"不该发生的情况"暴露出来;Sorted 与标准库 slices 协作,展示"自己造的工具与官方工具协同"的分工。改动建议:给 Set 加 Union/Intersect/Difference 三个集合运算(练习约束复用);给 Reduce 增加"无初始值版本"返回 (T, bool);把 Filter 改为原地过滤并对比语义差异。

下一章预告:第11章 反射与动态特性。泛型解决"编译期类型无关",反射解决"运行期类型自省":reflect 三法则、struct tag 读取、以及一个 40 行的迷你 JSON 编解码器——然后我们讨论性能与克制。